

UNIVERSO

Sistema de Agentes IA en UNIK

Documento técnico completo: arquitectura, agentes, herramientas, seguridad, modelos, costos y estado actual.

Generado: 25 de septiembre de 2026

Código: /src/modules/agents, /src/modules/ai, /src/modules/venues

Flag principal: UNIK_AGENT_RUNTIME_V2 (apagado por default)

1. Qué es UNIVERSO

UNIVERSO es la capa de agentes de UNIK. En vez de un solo copiloto que responde preguntas, hay un equipo de IA: un agente principal (UNIK Central, badge JEFE) que entiende al usuario, decide qué hacer, delega en especialistas, usa herramientas reales (base de datos, internet, navegador, computadora virtual, APIs) y reporta con evidencia.

El diseño no reescribe nada: envuelve el orquestador existente, sus ~220 herramientas, JEV (el decisor ligero), missions, venue y jobs. Todo corre detrás del flag UNIK_AGENT_RUNTIME_V2 y es aditivo en base de datos.

Principios rectores

- Un solo interlocutor: el usuario habla con el JEFE; los especialistas trabajan detrás.
- Honestidad dura: solo se afirma lo que una tool ejecutó de verdad en ese turno.
- Toda acción con efecto (enviar, pagar, borrar, publicar) pide aprobación humana.
- Barato por default: modelos chicos para rutina, modelos grandes solo cuando aportan.
- Observable: cada turno es un AgentRun con eventos, tokens y costo medido.

2. Arquitectura general

Flujo de un mensaje del usuario:

```
usuario !' POST /assistant/api/chat
! ' executeAgentTurn (agent-runtime)           [persona + allowlist + modelo del agente]
! ' JEV routing envelope                       [1 llamada: path, dominios, delegación, riesgo]
! ' runAssistant (orquestador)
! ' selectToolsForTurn                        [menú de tools por dominio]
! ' ToolGateway (policy paso 4d)              [grants, autonomía×efecto, presupuesto]
! ' tools: DB / web / venue / composio / archivos / media
! ' delegateTask !' AgentTask (DAG)           [workers en sesión fresca]
! ' AiProposal si el efecto lo requiere [aprobación humana]
! ' AgentRun + AgentEvent (traceId, tokens, costo)
! ' SSE: tokens, tool_call, workspace.*, agent.task, agent.message
! ' UI: chat + OpsPanel (pantalla viva, terminal, misiones, costo)
```

Piezas principales

Capa	Módulo	Qué hace
Identidad	agents/agent-service	Principal auto-creado por usuario, CRUD de especialistas, AgentVersion por cambio de persona.
Runtime	agents/agent-runtime	Inyecta persona al system prompt, intersecta tools con allowlist, aplica modelDefault.
Routing	ai/decisions/routing-envelope	JEV decide 10 cosas en UNA llamada (600 ms, fallback heurístico).
Delegación	agents/delegation + task-graph	delegateTask con cápsula "d8k, DAG con dependsOn, fan-out máx 4.
Ejecución	ai/tools/* (~220 tools)	DB, web, venue, Composio, documentos, media, memoria.
Gobernanza	agents/policy (ToolGateway)	Grants tool:/effect:, matriz autonomía×efecto, presupuesto por periodo.
Memoria	agents/memory-router	Scopes tenant!'user!'agent!'thread; modos full/on_demand/off.
Tiempo real	ai/workspace-events + SSE	Eventos workspace.* a la 3ª columna (pantalla, páginas, terminal).

Venue	venues/*	Computadora virtual desechable (Daytona) + navegador + exec + archivos.
Rutinas	agents/trigger-service	trigger.tick cada 60 s: time/condition/entity_change/webhook/playbook.

3. Los agentes

3.1 UNIK Central (JEFE)

Se crea solo la primera vez que el usuario abre el asistente (kind=principal, fijado arriba del sidebar). Tiene persona propia, acceso completo a las tools habilitadas, y es quien decide delegar. El usuario nunca coordina especialistas a mano.

3.2 Especialistas

Agentes creados por el usuario (NewAgentSheet) o por el JEFE vía delegación. Cada uno tiene: nombre, icono/color, persona (system prompt), toolAllowlist (subconjunto de tools), modelDefault, nivel de autonomía, venuePolicy y presupuesto. Editar la persona crea un AgentVersion (historial).

3.3 Workers transitorios

Cuando el JEFE delega, el worker corre en sesión fresca con una cápsula de contexto (objetivo + datos + restricciones, "d8k — nunca el historial completo). Al terminar reporta al padre por AgentMessage; el usuario ve el reporte plegado en el chat y la task en el terminal del OpsPanel.

3.4 Task DAG (task-graph)

- dependsOn entre tasks: una task no arranca hasta que sus dependencias cierran.
- Fan-out máximo 4 delegaciones por turno.
- Cancelar una task cancela en cascada sus descendientes.
- Cada task es un AgentRun hijo (linaje parentRunId/rootRunId — costos atribuibles).

4. JEV — el decisor ligero

JEV es un modelo chico y rápido que actúa como árbitro de metadatos, no como redactor. En cada turno resuelve el "routing envelope" en una sola llamada con deadline de 600 ms:

- path: respuesta directa, tools, RAG o delegación.
- domains: qué dominios toca (ventas, clientes, venue, web...).
- delegation: si conviene fan-out a especialistas.
- modelClass: simple | standard | complex | computer.
- risk: si el plan toca efectos que requieren aprobación.

Si JEV falla o expira, un fallback heurístico (regex de dominios + clasificador local) decide — el sistema nunca se queda sin router. La decisión queda en AgentEvent tipo "route" para métricas de costo de router.

Dónde NO entra JEV

JEV no lee pantallas ni escribe respuestas al usuario: clasifica y frena. Las acciones riesgosas las evalúa por paso (efecto read-equivalent vs external_send vs destructive) y el ToolGateway ejecuta la política.

5. Las 3 superficies de control

El agente puede actuar sobre tres superficies, todas visibles en el OpsPanel:

Superficie	Qué es	Estado
Computadora virtual	Sandbox Daytona desechable: navegador Chromium (browser), terminal (venueExec), filesystem, screenshots. Session con lease central y reaper de idle.	Implementado
Páginas web del agente	Páginas abiertas por tools web (fetch_url, web_search, web_crawl, browser). El panel las lista con contenido extraído.	Implementado
Apps por API	Toolkits de Composio conectados por el usuario (Gmail, Calendar, Slack, etc.) gobernados por políticas y roles.	Implementado
Computadora local (Mac)	App Electron con icono flotante + computer-use local vía cola de acts. Plan completo escrito.	Plan — no implementado

5.1 Computer use en la VM

El navegador corre dentro del sandbox: un browser-controller.mjs (node, puerto 3100) recibe acciones (open, click, type, scroll, extract, screenshot, pdf, tabs, secureInput...) y las ejecuta con playwright-core + chromium provisionados dentro de la imagen.

- Cada acción devuelve screenshot !' evento workspace.screen !' el panel muestra la página viva (chat izquierda, navegador real derecha).
- provision.sh instala node + playwright-core + chromium dentro del sandbox; los hosts de infra (npm, nodejs.org, debian,

Playwright CDN)
pasan aunque haya
allowlist de
dominios.

- El controller
escucha 0.0.0.0
con token x-unik-
token por request;
pkill antes de
respawn sana
sesiones viejas;
browserAct se auto-
cura una vez por
sesión.

6. Modelos y costo (O penRouter + OpenAI)

El model-router
asigna el modelo
más barato que
puede hacer el
trabajo. Elección
explícita del usuario
siempre gana.
Turnos de
computadora usan
el tier dedicado.

Tarea (AiTask)	Uso
simple	Saludos, confirmaciones
routine	Consultas del día a día
complex	Análisis, reportes
utility	Resúmenes, digests
judge	Juez de calidad de respuestas
vision	Extracción de imágenes
computer	Computer use: navegación

7. Herramientas (~220 registradas)

Familias principales

- Datos UNIK: querySalesOrders, universalSearch, getTopProducts, getSalesTrend, clientes, inventario, cotizaciones, facturas, pagos, paquetes, compras...
- Web: web_search (Tavily o Sonar-OpenRouter fallback), fetch_url, web_crawl, web_research.
- Venue: browser (open/click/type/extract/screenshot/pdf/secureInput), browserProfile (sesiones por sitio), venueExec, venueReadFile/WriteFile/ListFiles, venueScreenshot, VenuePlaybooks.
- Apps/API: tools de Composio por toolkit conectado (gateway externo con policies y roles).
- Documentos y media: composeDocument, artifacts, generación de imágenes/video/audio vía providers, attachments multi-tipo.
- Agentes: delegateTask, mensajes entre agentes, triggers/rutinas, memoria (remember/recall).

Selección por turno

selectToolsForTurn ofrece al modelo solo el subconjunto relevante (por dominios detectados + contexto de página), respetando enabledTools del admin y el toolAllowlist del agente. ContextTags acotan por superficie (chat, inbox, assistant...).

8. Seguridad, privacidad y aprobaciones

8.1 Gobernanza (ToolGateway — paso 4d de executeTool)

- AgentGrant permit/deny/require_approval por tool:x o effect:x.
- Matriz autonomía×efecto: read < write < external_send < destructive; autonomía del agente acota hasta dónde llega sin aprobación.
- Presupuesto por agente/periodo: se agota !' la acción se bloquea.

8.2 Aprobaciones

AiProposal es la tarjeta de aprobación. Las acciones con efecto externo (enviar, comprar, publicar, borrar, usar perfil guardado, credenciales) se pausan hasta que el usuario aprueba en el panel. El modelo declara intent=send/pay/purchase/publish/delete y el gateway decide.

8.3 Credenciales — secureInput

Cuando una página pide login/tarjeta, el modelo llama browser action=secureInput declarando los campos. El usuario los escribe en un formulario enmascarado de su panel; el controller los teclea directo en la página (useCredential). Los valores jamás pasan por el modelo, el chat ni la base de datos.

8.4 Pantallas — memory-only

Las capturas de pantalla son efímeras: viajan por SSE y por GET /venue/state (Cache-Control: no-store) y mueren ahí. stripScreenData las quita del audit (AiToolCall.result), del mensaje tool persistido y del contexto del modelo — para "ver" la pantalla el modelo usa analyzeImage (resultado acotado), nunca el base64.

8.5 Datos no confiables

Resultados de web_search, fetch_url, web_crawl, browser y venue* llegan envueltos en <untrusted>: son datos, nunca instrucciones. Prompt injection en páginas se reporta, no se obedece.

8.6 Kill switches

- UNIK_VENUE_ENABLED=false apaga toda la capa venue.
- venueEnabled / webSearchEnabled / webFetchEnabled en Admin !' Asistente.
- UNIK_AGENT_RUNTIME_V2 apaga el runtime multiagente completo.
- Roles/permisos por tool (p. ej. browser.use) y allowedRoleKeys en tools externas.

9. Memoria

memory-router organiza la memoria en scopes con modo por agente:

Scope	Qué guarda	Quién lo ve
tenant	Hechos de la empresa (políticas, datos de negocio).	Todos los agentes del tenant
user	Preferencias y hechos del usuario.	Agentes de ese usuario
agent	Conocimiento propio del agente (su rol, aprendizajes).	Ese agente
thread	Contexto del hilo/conversación.	Esa conversación

- Modos: full (inyecta al prompt), on_demand (solo vía tools — workers), off.
- AiMemory persistido con tenantId/agentId/conversationId — sin fugas cross-tenant.

10. Rutinas, triggers y misiones

10.1 Triggers (always-on)

trigger.tick corre cada 60 s y evalúa triggers del usuario:

- time — horarios/cron ("cada lunes 8am").
- entity_change — cuando cambia una entidad vigilada (orden, cliente).
- condition — sondas deterministas sin LLM (tools read-only whitelisted); solo despierta agente si hay novedad.
- webhook — disparo externo.
- playbook — corre un

VenuePlaybook grabado sin LLM: repetición exacta de un flujo web aprendido una vez.

10.2 Misiones

Trabajos largos con objetivo, tasks (DAG) y progreso vivo. MissionCard muestra tasks en curso; el terminal del OpsPanel enlista agent.task en tiempo real. Las misiones pueden correr en segundos planos y reportan al terminar.

11. Observabilidad

y costo

- AgentRun por turno/delegación: traceld, linaje, modelo, tokens in/out, costo estimado.
- AgentEvent: journal del run (routing envelope, cada tool con duración, errores).
- VenueSession: minutos facturados (billedMinutes) !' UsageMeter dim "venue".
- GET /assistant/api/usage !' {llm, venue, jev, runs, spent} medido real.
- OpsPanel: SUPERFICIES (3), TERMINAL VIVO (tools en curso), pantalla venue, misiones, vigilancias, aprobaciones pendientes, costo del equipo.

12. APIs del sistema de agentes

Ruta	Contenido
GET/POST /app/assistant/api/agents	Lista del equipo / crear especialista
PATCH/DELETE /agents/[id]	Editar (crea AgentVersion) / archivar (principal no se archiva)
POST /chat	Turno del asistente; acepta agentId (fija el agente del hilo)
GET /runs/[id]	Run + eventos + DAG de tasks
POST /tasks/[id]/cancel	Cancelar task + cascada
GET/POST/PATCH/DELETE /triggers	Rutinas
GET /workspace	Workspace central + lease actual
GET /usage	Costos medidos {llm, venue, jev, runs, spent}
GET /venue/state	Pantalla viva + secure-inputs pendientes (no-store)
POST /venue/secure-input	El usuario envía valores enmascarados al controller

13. Frontend

- AgentSidebar — "Tu equipo" (JEFE fijado), misiones recientes, "Descargar app" (PWA).
- OpsPanel — 3ª columna: superficies, pantalla viva, terminal, misiones, vigilancias, aprobaciones, costo.
- AssistantWidget — FAB flotante con el chat del agente en todas las páginas excepto /app/assistant.
- MissionCard / ActivityCard / AgentMessageCard / RoutineChip / NewAgentSheet / TweaksPanel / AgentAvatar.
- meta.agent (avatar por respuesta), meta.agentMessages (fold "Mensajes de X"), meta.routineCreated (chip).
- SSE: workspace.* en assistant:{conversationId}; agent.task/agent.message en user:{id}.

14. Configuración

Variables de entorno

Variable	Qué hace
UNIK_AGENT_RUNTIME_V2	Activa el runtime multiagente (default apa
UNIK_VENUE_ENABLED	Kill-switch global de venues.
DAYTONA_API_KEY / API_URL / TARGET	Proveedor de la computadora virtual.
UNIK_COMPUTER_MODEL	Override del modelo de computer use.
OPENROUTER_API_KEY	Habilita el catálogo OpenRouter (modelos
TAVILY_API_KEY	Búsqueda web primaria (sin ella cae a So

Settings en Admin !' Asistente

providerConfigs
(llaves por proveedor),
reparto de modelos
por tarea (incluido
computerUseModel),
enabledTools,
venueEnabled,
venueImage, límites
de venue,
webDomainAllowlist/
Denylist,
webSearchEnabled,
guardrails, rate limits.

15. Estado actual (honesto)

Implementado y en main

- Runtime multiagente completo (identidad, delegación, DAG, triggers, gateway, memoria, tenancy semilla).
- Venue Daytona: exec, archivos, navegador con screenshots vivos, secureInput, perfiles de sesión, playbooks.
- OpsPanel con pantalla en vivo, terminal, superficies, costos; widget flotante; PWA instalable.
- Router con tier computer dedicado (Gemini Flash vía OpenRouter) y fallback heurístico de JEV.
- Privacidad de pantallas: efímeras, stripScreenData, no-store.
- Búsqueda web con fallback Sonar vía OpenRouter.

Pendiente / limitaciones conocidas

- Multiempresa duro: tenantId existe en todo el schema pero el resolver cae a "unik" hasta que haya una segunda empresa real.
- Job worker corre dentro del proceso web (durable pero comparte proceso); separación prevista cuando haya carga.
- Evals automáticos de routing/seguridad definidos en el plan, aún no escritos.
- App de escritorio Mac + computadora local: plan completo aprobado (Electron, bubble flotante, acts por cola, LocalVenue como 4º provider) — sin implementar.
- Verificación end-to-end real en producción: continua y manual (venue, browser, search, aprobaciones).

16. Glosario

JEV: Decisor ligero (routing envelope, clasificaciones, gating de riesgo).

Venue: Computadora desechable del agente (Daytona hoy; Mac local en plan).

AgentRun / AgentEvent: La grabación de cada turno con linaje, tokens y costo.

AgentTask: Unidad de delegación dentro del DAG (cápsula + dependsOn).

Workspace / Lease: La venue central compartida con candado por run.

ToolGateway: La puerta que autoriza cada tool por grants/efecto/presupuesto.

secureInput: Canal para que el usuario teclee secretos directo a la página.

Playbook: Flujo web grabado que se repite determinísticamente sin LLM.

OpsPanel: Tercera columna: superficies, pantalla viva, terminal, costos.

stripScreenData: Sanitizador que expulsa capturas del audit/historial/contexto.